Solution: Find Original Array From Doubled Array

Problem Statement

A doubled array `changed` is formed from the `original` array after appending twice the value of each element in the `original` array and `randomly shuffling` elements in the updated `original` array.

Given an array `changed`, return the `original` array if `changed` is a valid `doubled` array. Otherwise, return `empty` array.

Examples

- **Example 1**:
  - **Input**: `changed = [2,4,1,8]`
  - **Expected Output**: `[1,4]`
  - **Justification**: The original array [**1,4**] is doubled to form [**2,4,1,8**] (1's double is 2, and 4's double is 8).
- **Example 2**:
  - **Input**: `changed = [6,3,0,0]`
  - **Expected Output**: `[0,3]`
  - **Justification**: The original array [**0,3**] is doubled to form [**6,3,0,0**] (0's double is 0, and 3's double is 6).
- **Example 3**:
  - **Input**: `changed = [1, 2, 2, 4, 4, 8, 16, 32]`
  - **Expected Output**: `[1, 2, 4, 16]`
  - **Justification**: The original array [**1, 2, 4, 16**] is doubled to form [**1, 2, 2, 4, 4, 8, 16, 32**] (1's double is 2, 2's double is 4, 4's double is 8, and 16's double is 32).

Solution

To solve this problem, the approach begins with sorting the given array. Sorting helps in easily identifying the pairs of original elements and their doubled counterparts. After sorting, we'll use a hashmap or a frequency counter to keep track of the occurrences of each element in the array. This is crucial for handling duplicates effectively.

We iterates through the sorted array, and for each element, we check if its double exists in the hashmap while ensuring there's a remaining count for both the element and its double. If such pairs are found, the element is added to the result array, and the counts are updated accordingly. This method works because sorting and counting allow us to systematically pair each element with its double, ensuring we reconstruct the original array accurately and handle cases with duplicates or zeros effectively.

Step-by-step Algorithm

- Sort the given array in non-decreasing order.
- Initialize a hashmap (or frequency counter) to keep track of the occurrences of each element in the array.
- Iterate through the sorted array:
  - For each element `x`, check if the element `2*x` exists in the hashmap and has a non-zero count.
  - If such a pair is found, add `x` to the result array and decrement the counts for `x` and `2*x` in the hashmap.
  - If `x` is zero, special handling is required to ensure pairs of zeros are correctly accounted for.
- Continue the process until all valid pairs are identified and the corresponding elements are added to the result array.
- If at any point, a required pair cannot be found, it means the original array cannot be reconstructed, and an empty array should be returned.

Algorithm Walkthrough

Taking the input [**1, 2, 2, 4, 4, 8, 16, 32**]

1. **Check if the array's length is even**: Since the array has 8 elements, we proceed because it's even.
2. **Sort the array**: The array is already in non-decreasing order: [**1, 2, 2, 4, 4, 8, 16, 32**].
3. **Initialize a frequency counter** (**HashMap or Dictionary**): We count the occurrences of each element in the array.

```
{
  1: 1,
  2: 2,
  4: 2,
  8: 1,
  16: 1,
  32: 1
}
```

4. **Iterate through the array**: Starting with the first element, we look for its double and check if we can form a valid pair by decrementing counts in the frequency counter.
   - **For 1**: Its double is 2. We find 2 in the frequency counter, decrement the count for 1 and 2.
     - Updated frequency counter:

       ```
       {
         1: 0,
         2: 1,
         4: 2,
         8: 1,
         16: 1,
         32: 1
       }
       ```

     - Add 1 to the result array.
   - **For the next 2 (first 2 encountered)**: Its double is 4. We find 4 in the frequency counter, decrement the count for 2 and 4.
     - Updated frequency counter:

       ```
       {
         1: 0,
         2: 0,
         4: 1,
         8: 1,
         16: 1,
         32: 1
       }
       ```

     - Add 2 to the result array.
   - **Skip the second 2 since its count is now 0.**
   - **For the first 4 encountered**: Its double is 8. We find 8 in the frequency counter, decrement the count for 4 and 8.
     - Updated frequency counter:

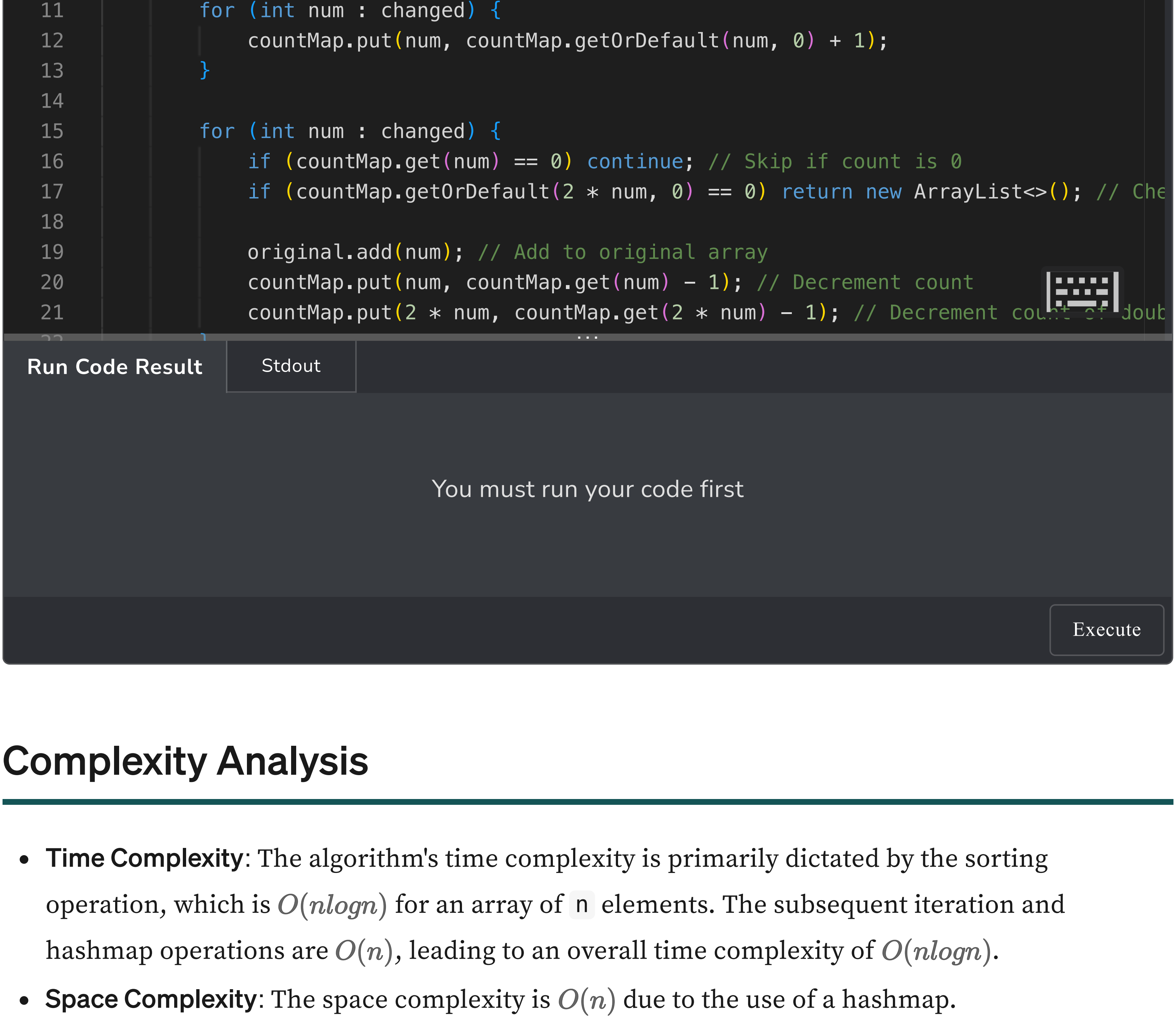
       ```
       {
         1: 0,
         2: 0,
         4: 0,
         8: 0,
         16: 1,
         32: 1
       }
       ```

     - Add 4 to the result array.
   - **Skip the second 4 since its count is now 0.**
   - **Skip the 8 since its count is now 0.**
   - **For 16**: Its double is 32. We find 32 in the frequency counter, decrement the count for 16 and 32.
     - Updated frequency counter:

       ```
       {
         1: 0,
         2: 0,
         4: 0,
         8: 0,
         16: 0,
         32: 0
       }
       ```

     - Add 16 to the result array.
5. **Result Array**: By the end of the iteration, the result array is [**1, 2, 4, 16**], which matches our expectation based on the input array.

Code



```java
import java.util.*;

public class Solution {
    public List<Integer> findOriginalArray(int[] changed) {
        if (changed.length % 2 != 0) return new ArrayList<>(); // Array size must be eve
        Arrays.sort(changed); // Sort the array
        Map<Integer, Integer> countMap = new HashMap<>();
        List<Integer> original = new ArrayList<>();

        // Count occurrences of each element
        for (int num : changed) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }

        for (int num : changed) {
            if (countMap.get(num) == 0) continue; // Skip if count is 0
            if (countMap.getOrDefault(2 * num, 0) == 0) return new ArrayList<>(); // Che

            original.add(num); // Add to original array
            countMap.put(num, countMap.get(num) - 1); // Decrement count
            countMap.put(2 * num, countMap.get(2 * num) - 1); // Decrement count of doub
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The algorithm's time complexity is primarily dictated by the sorting operation, which is $O(nlogn)$ for an array of `n` elements. The subsequent iteration and hashmap operations are $O(n)$, leading to an overall time complexity of $O(nlogn)$.
- **Space Complexity**: The space complexity is $O(n)$ due to the use of a hashmap.

← Previous
Find Original Array From Doubled Array (Medium)

Next →
Reverse Words In A String II

☐ Mark as Completed